



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Modelo, niveles de modelado, metamodelo, DSL y MDD aplicados a SVIF

Material puente hacia las Semanas 3 (CIM) y 4 (PIM → PSM):
cómo el caso SVIF instancia cada nivel conceptual.

Magíster en Ingeniería Informática · UFRO

EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

¿Qué es un modelo? Instanciado en SVIF

Un modelo es una **representación de la realidad para algún propósito definido** (Pidd, 2003). Entre el sistema y el modelo media la relación *is-represented-by*.

Todo modelo se construye conforme a un **paradigma de modelado** (Aßmann, Zschaler y Wagner, 2006).

En SVIF esto se concreta así:

- Antes de escribir una línea de C++, el sistema queda **representado por modelos**.
- Permiten **razonar sobre objetivos en conflicto** (privacidad ↔ oportunidad de la respuesta).
- Documentan la arquitectura de dos nodos y comunican decisiones al equipo.

CALIDADES DE UN MODELO ÚTIL (SELIC, 2003)

5 atributos

1

Abstracto

2

Comprensible

3

Preciso

4

Predictivo

5

Barato

(significativamente más barato que construir el sistema)

Niveles de modelado en el proyecto SVIF

NIVEL	ESPACIO VISUAL (SVIF)	EJEMPLO EN EL PROYECTO
M0 · SISTEMA	Recinto con cámara y cerradura	ESP32-CAM vigilando la puerta
M1 · MODELO	cim-istar-svif.drawio, pim-dsl-svif.drawio	El agente <i>Face Monitor Component</i> con su objetivo «Monitorear presencia de personas»
M2 · METAMODELO	iStar 2.0 (Dalpiaz, Franch y Horkoff, 2016); metamodelo del DSL PIM del curso	Reglas: una <i>dependency</i> conecta depender–dependum–dependee; una <i>OnIntervalAction</i> posee <i>interval_in_milliseconds</i>
M3 · METAMETAMODELO	XML de diagrams.net (mxGraph)	Los modelos se serializan como XML y por ello son transformables con XSLT

La elección de XML como formato serializado (mxGraph) es lo que hace viable transformar CIM → PIM con **XSLT** y PIM → PSM con scripts Python.

¿Por qué un DSL y no solo UML?

UML describe bien clases, objetos, componentes e interacciones, pero el dominio CPS incorpora conceptos específicos:

- HILOS PERIÓDICOS
- RECURSOS DE HARDWARE
- MENSAJES ENTRE NODOS FÍSICOS

Representarlos con extensiones UML incrementaría la complejidad del modelado.

En SVIF, el DSL permite expresar:

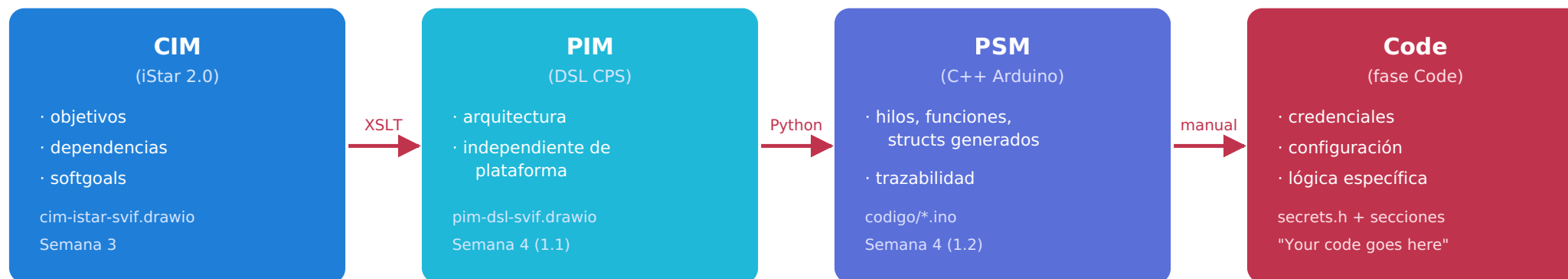
«Identificar un rostro cada **500 ms** y publicar el evento»
— sin comprometerse aún con una plataforma concreta.

TRES ELEMENTOS DEL DSL PIM DEL CURSO

En SVIF

ELEMENTO	DSL PIM PARA CPS
Sintaxis abstracta	CPCoordinate, OnIntervalAction, OnDemandAction, SWResource, HWResource, MessageSender, MessageReceiver, operadores AND/OR
Sintaxis concreta	Biblioteca <i>scratchpad PIM-DSL</i> para diagrams.net
Semántica	Traducción hacia hilos FreeRTOS, funciones C++, structs y comentarios de trazabilidad en Arduino

MDD aplicado: la cadena concreta del caso



Cada transformación incorpora decisiones de diseño del siguiente nivel, preservando la trazabilidad mediante identificadores (id_cim_parent).

Lo nuevo en cada nivel (lo que *no* estaba antes)

ETAPA	DECISIONES INCORPORADAS POR EL DISEÑADOR
CIM → PIM	Períodos (500 ms / 250 ms); parámetros E/S de las OnDemandAction; estructura del <i>dependum</i> (timestamp, person_id, person_name, confidence, authorized); campos de los SWResource.
PIM → PSM	Plataforma objetivo: ESP32 (Arduino core) . Tecnología de comunicación: MQTT . Tipado concreto (unsigned long, char[32], float, bool). Pines (RELAY_PIN, BUZZER_PIN).
PSM → CODE	Credenciales WiFi y broker; umbral de confianza; política de autorización; SIMULATION_MODE; duraciones de desbloqueo/alarma; tópico MQTT exacto.

Idea clave: cada nivel agrega información **no derivable** automáticamente del anterior. El valor del MDD es postergar la decisión al nivel donde corresponde.

Qué deja Semana 2 para SVIF

Conceptos instalados

- El sistema se **representa** con modelos antes de existir como código.
- Cada nivel (M0–M3) tiene un **rol distinto** en la pirámide.
- Cuando UML no alcanza, se construye un **DSL** con metamodelo + notación + semántica.
- **MDD** es la cadena de transformaciones que lleva del CIM al código.
- Cada transformación **agrega decisiones** y preserva trazabilidad.

Referencias:

- Aßmann, U., Zschaler, S., & Wagner, G. (2006). *Ontologies, meta-models, and the model-driven paradigm*. Springer.
- Bézivin, J. (2005). *On the unification power of models*. *Software & Systems Modeling*, 4(2), 171–188.
- Dalpiaz, F., Franch, X., & Horkoff, J. (2016). *iStar 2.0 language guide*. *arXiv:1605.07767*.
- Fowler, M., & Parsons, R. (2010). *Domain-Specific Languages* (Addison-Wesley Signature Series). Addison-Wesley.
- Mernik, M., Heering, J., & Sloane, A. M. (2005). *When and how to develop domain-specific languages*. *ACM Computing Surveys*, 37(4), 316–344.
- Pidd, M. (2003). *Tools for Thinking: Modelling in Management Science* (2nd ed.). Wiley.
- Selic, B. (2003). *The pragmatics of model-driven development*. *IEEE Software*, 20(5), 19–25.